

1. Explicación general del sistema

Mi sistema es una aplicación web para la reserva de entradas a conciertos masivos. Permite que un usuario inicie sesión, visualice conciertos disponibles, seleccione un tipo de entrada, haga una reserva y registre el pago. También tiene un perfil de administrador que puede ver información general del sistema, como entradas vendidas, disponibles e ingresos.

2. Tecnologías usadas

El sistema está desarrollado con HTML, CSS y JavaScript para la interfaz. Para el servidor uso Python con Flask. La base de datos está en MySQL, donde se almacenan usuarios, conciertos, entradas, reservas y pagos.

Archivos relacionados:

index.html -> estructura de la interfaz
styles.css -> diseño visual
app.js -> lógica del navegador
app.py -> servidor Flask
database.py -> conexión con MySQL
schema.sql -> estructura de la base de datos
modules/ -> rutas separadas del backend

3. Cómo funciona el flujo

Primero el usuario inicia sesión. El sistema valida el correo y la contraseña en la tabla usuario. Después carga los conciertos desde la tabla concierto y sus entradas desde la tabla entrada. Cuando el usuario reserva, se crea un registro en la tabla reserva. Si realiza el pago, se guarda en la tabla pago y se actualiza la cantidad de entradas vendidas.

4. Cómo explicar la base de datos

La base de datos se llama reservas_conciertos. Tiene cinco tablas principales:

usuario -> guarda los usuarios y administradores
concierto -> guarda los eventos disponibles
entrada -> guarda tipos de entrada, precio, total y vendidas
reserva -> guarda las reservas realizadas por usuarios
pago -> guarda los pagos asociados a cada reserva

Relaciones:

Un usuario puede hacer muchas reservas. Un concierto puede tener varios tipos de entrada. Una reserva pertenece a un usuario, a un concierto y a una entrada. Un pago pertenece a una reserva.

5. Cómo explicar app.py

app.py es el archivo principal del servidor. Aquí se crea la aplicación Flask, se cargan las variables del archivo .env, se registran los módulos del sistema y se indica que el sistema corre en el puerto 3000.

También:

Este archivo sirve tanto para mostrar el index.html como para responder las peticiones de la interfaz mediante rutas /api.

Ejemplo:

http://localhost:3000/	-> muestra la interfaz
/api/login	-> inicia sesión
/api/concerts	-> devuelve conciertos
/api/reservations	-> maneja reservas
/api/payments	-> maneja pagos

. Cómo explicar database.py

database.py contiene la conexión a MySQL. Lee los datos desde .env, como usuario, contraseña, puerto y nombre de la base de datos. Además tiene funciones reutilizables para ejecutar consultas SQL.

Ejemplo de configuración:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=reservas_user
DB_PASSWORD=123456
DB_NAME=reservas_conciertos
```

7. Cómo explicar la carpeta modules

La carpeta modules divide el backend por funcionalidades. Esto ayuda a que el código esté más ordenado.

usuarios.py	-> login y registro
conciertos.py	-> listar y crear conciertos
reservas.py	-> crear, listar y cancelar reservas
pagos.py	-> registrar pagos

8. Cómo explicar app.js

app.js controla la interacción de la página. Se encarga de enviar peticiones al servidor, mostrar conciertos, manejar la selección de entradas, crear reservas, procesar pagos y actualizar la interfaz sin recargar toda la página.

El frontend no accede directamente a MySQL. Siempre se comunica con Flask usando rutas /api.

9. Explicación importante: por qué se usa localhost:3000

El sistema debe abrirse desde `http://localhost:3000` porque necesita el servidor Flask. Si abro solo `index.html`, la interfaz carga, pero no puede conectarse a /api ni a la base de datos.

`index.html` es la pantalla, `app.py` es el servidor y MySQL es donde están los datos.

10. En conclusión

el sistema permite gestionar reservas de entradas para conciertos, conectando una interfaz web con un servidor Flask y una base de datos MySQL. La aplicación está dividida por módulos para separar usuarios, conciertos, reservas y pagos, lo que hace que el código sea más ordenado y fácil de mantener.

Explicación general

Este archivo `app.js` controla toda la interacción de la interfaz. No se conecta directamente a MySQL, sino que se comunica con el servidor Flask mediante rutas que empiezan con `/api`. Desde aquí se maneja el inicio de sesión, la carga de conciertos, la selección de entradas, la creación de reservas, el pago y la visualización del perfil.

1. Variables principales

```
js
const API_URL = "/api";
let currentUser = null;
let concerts = [];
let reservations = [];
let selectedConcert = null;
let selectedTicket = null;
```

Puedes explicar:

Aquí se guardan los datos temporales que usa la interfaz. `currentUser` guarda el usuario que inició sesión, `concerts` guarda los conciertos recibidos desde la base de datos, `reservations` guarda las reservas, y `selectedConcert` / `selectedTicket` guardan lo que el usuario está seleccionando.

2. Comunicación con el backend

```
js
async function api(path, options = {}) {
  const response = await fetch(`${API_URL}${path}`, ...)
}
```

Explicación:

Esta función centraliza todas las peticiones al servidor. Por ejemplo, si quiero iniciar sesión, llama a `/api/login`. Si quiero traer conciertos, llama a `/api/concerts`. Además envía el ID del usuario en los encabezados para que el backend sepa quién está usando el sistema.

3. Formato de datos

```
js
formatMoney()
formatDate()
formatTime()
```

Explicación:

Estas funciones solo convierten los datos a un formato más entendible para el usuario. Por ejemplo, muestran el precio en dólares y la fecha en formato de Ecuador.



4. Control de pantallas

```
js
function showView(name) { ... }
```

Explicación:

Esta función cambia entre las diferentes pantallas del sistema: login, conciertos, selección de entradas, resumen, pago, confirmación, reservas y perfil. En vez de recargar la página, muestra u oculta secciones del HTML.

5. Cargar y mostrar conciertos

```
js
function renderConcerts() { ... }
```

Explicación:

Esta función toma la lista de conciertos obtenida desde la base de datos y la dibuja en la pantalla. También permite filtrar por el buscador y muestra el precio más bajo de cada concierto.

Puedes decir:

Aquí no se crean los conciertos; solo se muestran. Los datos vienen desde Flask y MySQL.

6. Selección de entradas

```
js
function openTicketSelection(concertId) { ... }
function renderTicketSelection() { ... }
```

Explicación:

Cuando el usuario elige un concierto, el sistema guarda ese concierto seleccionado y muestra los tipos de entrada disponibles, como General o VIP. También calcula el total según la cantidad seleccionada.

7. Crear reserva

```
js
async function createReservationFromSelection() { ... }
```

Explicación:

Esta función se ejecuta cuando el usuario presiona Continuar. Envía al backend el concierto, el tipo de entrada y la cantidad. El servidor crea la reserva en la tabla `reserva` y devuelve un ID de reserva.

Frase útil:

La reserva no se guarda en el navegador; se guarda en MySQL mediante el backend.

8. Mostrar reservas

```
js
function renderReservations() { ... }
```

Explicación:

Esta función muestra las reservas del usuario. Si una reserva está activa, permite pagarla o cancelarla. Si ya fue pagada, aparece como confirmada.

9. Pago

```
js
async function processPayment(event) { ... }
```

Explicación:

Esta función procesa el pago. Envía la reserva y el método de pago al backend. Si el pago es aprobado, se actualiza la reserva como confirmada y se registra el pago en la tabla `pago`.

10. Perfil y administrador

```
js
function renderSession() { ... }
function renderAdmin(summary) { ... }
```

Explicación:

`renderSession` muestra los datos del usuario en el perfil. `renderAdmin` se usa cuando el usuario es administrador y muestra métricas como entradas vendidas, bloqueadas, disponibles e ingresos.

11. Login y registro

```
js
loginForm.addEventListener("submit", ...)
registerForm.addEventListener("submit", ...)
```

Explicación:

Estas partes escuchan cuando el usuario envía el formulario de login o registro. Luego llaman a `/api/login` o `/api/register`. Si el servidor responde correctamente, se guarda el usuario en `sessionStorage`.

12. Guardar sesión

```
js
sessionStorage.setItem("currentUser", JSON.stringify(currentUser));
```

Explicación:

Esto permite que el usuario siga conectado aunque se recargue la página. La sesión se guarda temporalmente en el navegador.

13. Actualización automática

```
js
window.setInterval(() => {
  if (currentUser) refreshData(false);
}, 30000);
```

Explicación:

Cada 30 segundos el sistema actualiza los datos para mantener las reservas, disponibilidad e inventario al día.

En resumen, `app.js` es el archivo que maneja la lógica visual del sistema. Se encarga de escuchar los clics del usuario, llamar al backend Flask mediante `/api`, recibir los datos de MySQL y actualizar la pantalla. Gracias a este archivo, el sistema puede iniciar sesión, mostrar conciertos, crear reservas, registrar pagos y mostrar información del perfil sin recargar toda la página.